

EXPLICATION IMPLEMENTATION FONCTION NEGATE

PRINCIPE

Un DFA est un automate qui, à la fin d'un mot, dit :

-ACCEPTE

Ou

-REJETÉ.

Alors la négation du DFA, sera une nouvelle machine qui fait l'inverse :

➔ Ce qu'il acceptait → maintenant rejeté

➔ Ce qu'il rejetait → maintenant accepté

Comment faire :

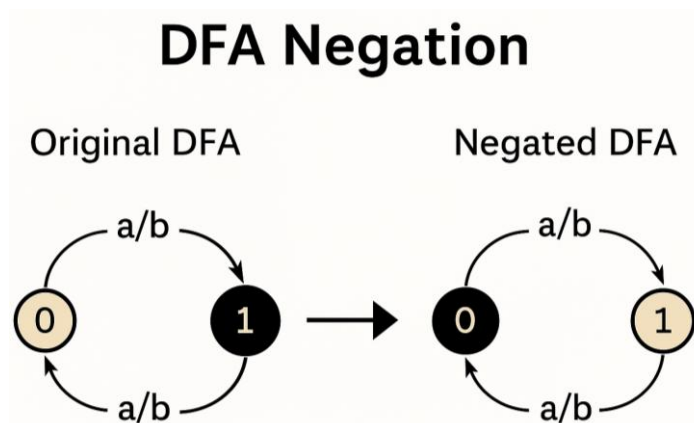
On se base sur la complémentarité d'un Langage régulier à partir de son automate

soit on a un langage L qui est reconnu par un **DFA A** alors son complémentaire L^c est simplement reconnu par un **DFA B** en inversant les états finaux.

-On garde tout indentique (l'entrée , l'etat initial , delta)

-Mais on change : etat final en non final et etat non final en final.

DEMONSTRATION



L'image montre:

- à gauche : un DFA original,
- à droite : le DFA négatif (complément).

1 Partie gauche : “Original DFA”

Dans ce DFA :

- Il y a deux états : 0 et 1.
- L'état 0 est représenté en *beige clair* → c'est un état final (acceptant).
- L'état 1 est en noir → non final (rejetant).

Les flèches montrent les transitions :

- lire 'a' ou 'b' amène de 0 vers 1,
- lire 'a' ou 'b' amène aussi de 1 vers 0.

Interprétation simple :

➔ *Le DFA original accepte les mots qui se terminent dans l'état 0.
Et il rejette les mots qui se terminent dans l'état 1.*

2 Partie droite : “Negated DFA”

Dans ce DFA négatif :

- Les transitions sont identiques (rien n'a changé dans la structure).
- MAIS les couleurs ont changé :
 - l'état 0 est maintenant noir → non-final
 - l'état 1 est maintenant beige → final

Autrement dit :

Tous les états finaux deviennent non-finaux et tous les non-finaux deviennent finaux.

A RETENIR :

Avant négation :

- si la machine finissait en état 0 → accepté
- si elle finissait en état 1 → refusé

Après négation :

- si la machine finit en état 0 → refusé
- si elle finit en état 1 → accepté

ANALYSE DE LA LA FONCTION

```
(** DFA Negation *)  
val negate : 's t -> 's t
```

```
let negate dfa =  
  let finals =  
    List.filter (fun s -> not (List.mem s dfa.finals)) dfa.states  
  in  
  { dfa with finals }
```

- Cette fonction prend en entrée un DFA (dfa : 's t) et retourne un nouveau DFA
- Ensuite pour effectuer la négation on construit une liste qui contient les états qui sont non finaux :

List.filter (fun s -> not (List.mem s dfa.finals)) dfa.states

dfa.states = tous les états du DFA

dfa.finals = les états finaux actuels

List.filter : garde les états qui valident un prédicat.

Donc ici le prédicat c'est de savoir si :

s est un état final ? : **List.mem s dfa finals** , si oui on inverse : **not(...)**

exemple avec le tableau :

S	List.mem s finals	Not(...)	Gardé
0	False	True	Oui
1	True	False	Non

=> finals=[0]

-Enfin on crée notre nouveau DFA avec l'ensemble des états finaux inversés

{ dfa with finals = new_finals }

CONCLUSION

Avec notre fonction on crée un nouveau DFA qui :

- a le même alphabet
- a les mêmes états
- a la même fonction de transition
- a le même état initial
- a les états finaux inversés